

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ**
федеральное государственное бюджетное образовательное учреждение
высшего образования
**«УЛЬЯНОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ
УНИВЕРСИТЕТ»**

Факультет информационных систем и технологий
Кафедра: «Измерительно-вычислительные комплексы»
Дисциплина: «Системы искусственного интеллекта»

Отчёт
По лабораторной работе №6

Выполнила:
студент гр. ИСТбд-32
Минибаева Е.Р.
Проверил:
к. т. н., доцент каф. ИВК
Шишкин В.В.

Ульяновск
2026 г.

Введение

В рамках данной лабораторной работы была поставлена задача разработать программную реализацию игрового агента, обучающегося методом обучения с подкреплением без учителя, для настольной игры «Шашки Артамонова». Цель работы заключалась в том, чтобы на практике сравнить два подхода к построению игрового искусственного интеллекта — традиционный переборный метод Минимакс с альфа-бета отсечением и нейросетевой агент, обучающийся на собственном опыте, — а также продемонстрировать возможность превзойти статически заданную стратегию за счёт машинного обучения.

В работе реализован графический интерфейс, позволяющий человеку играть как против минимаксного бота, так и против обученного RL-агента, а также предусмотрен режим тренировки последнего.

Теоретическая часть

Минимакс и альфа-бета отсечение.

Минимакс представляет собой рекурсивный алгоритм поиска по игровому дереву для игр двух лиц с нулевой суммой. На каждом уровне дерева ходы делает один из противников, и алгоритм поочередно максимизирует и минимизирует оценочную функцию, предполагая оптимальную игру обеих сторон. Глубина поиска определяет, на сколько полуходов вперёд просчитываются варианты; в данной реализации минимаксный бот использует глубину четыре, то есть видит на два полных хода каждой стороны. Для ускорения работы применяется альфа-бета отсечение, позволяющее отбрасывать ветви, гарантированно не влияющие на итоговый выбор корневого узла. Эффективность отсечений повышается предварительной сортировкой ходов, при которой взятия рассматриваются в первую очередь. Статическая оценочная функция, вызываемая в листьях дерева или по достижении терминального состояния, учитывает разницу захваченных фигур, а также мобильность — количество доступных ходов. Минимакс не обладает способностью к обучению и действует строго в соответствии с заложенными человеком правилами оценки позиции.

Обучение с подкреплением и глубокая Q-сеть (DQN).

Обучение с подкреплением — это метод, в котором агент учится принимать решения, взаимодействуя со средой и получая числовую награду. Задача формулируется как марковский процесс принятия решений, а цель агента — максимизировать ожидаемую суммарную дисконтированную награду. Q-функция определяет ценность выполнения конкретного действия в заданном состоянии при условии дальнейшего оптимального поведения. Поскольку пространство состояний в шашках слишком велико для табличного представления Q-значений, применяется аппроксимация с помощью искусственной нейронной сети. Это приводит к архитектуре глубокой Q-сети (DQN). В отличие от табличного Q-обучения, нейросеть способна обобщать и давать осмысленные оценки даже для ранее не встречавшихся позиций.

Предобучение имитацией и self-play.

Прямое обучение DQN с нуля в играх часто страдает от медленной сходимости. Для ускорения используется двухэтапный подход. На первом этапе агент обучается на данных, сгенерированных экспертом — минимаксным ботом с глубиной 4. Нейросеть предобучается предсказывать высокое Q-значение для действий, выбранных учителем, что фактически является регрессией к константе. На втором этапе запускается самостоятельная игра (self-play), в которой агент играет против своей копии или случайного соперника, собирает опыт в буфер воспроизведения и обновляет веса методом временной разности (TD). Награда формируется по исходам партий и за взятие фигур. При выборе действия используется ϵ -жадная стратегия, обеспечивающая баланс между исследованием новых ходов и использованием уже известных сильных продолжений. Буфер воспроизведения является динамической структурой фиксированного объёма, хранящей последние переходы и постоянно обновляющейся по мере поступления нового опыта.

Ход работы

Работа выполнялась в несколько этапов.

1. Разработка Минимакс-бота. Реализован класс MinimaxBot с настраиваемой глубиной поиска (по умолчанию 4). Алгоритм использует рекурсивный обход дерева с альфа-бета отсечением и сортировкой ходов. Для симуляции ходов применяется глубокое копирование состояния доски. Бот интегрирован в графический интерфейс и может выступать соперником для человека.

2. Создание RL-бота и его обучение. Реализован класс RLBot, содержащий полносвязную нейронную сеть с архитектурой 68–256–128–1 (вход — конкатенация векторов состояния и действия, выход — Q-значение). Обучение разделено на два последовательных процесса. Сначала функция `generate_imitation_data` запускает 100 партий между двумя минимаксными ботами и собирает пары «состояние – действие эксперта». Затем `pretrain_rl_bot` выполняет предобучение сети методом градиентного спуска, подгоняя выход сети к значению 1.0 для действий учителя. После этого запускается `train_rl_selfplay`, где агент проводит 300 самостоятельных игр против собственной копии, пополняя буфер воспроизведения и периодически обновляя веса по TD-ошибке. Все операции обучения выполняются в фоновом потоке, чтобы интерфейс оставался отзывчивым. Прогресс отображается в главном меню.

3. Интеграция и тестирование. Обученный RL-бот сохраняется в файл и в дальнейшем может быть загружен для игры против человека. Проведено визуальное сравнение качества игры: после двухэтапного обучения RL-агент демонстрирует способность стабильно побеждать минимаксного бота с глубиной 4. Бот с подкреплением не осуществляет перебор вариантов, но благодаря обученной сети мгновенно оценивает легальные ходы и выбирает наилучший с точки зрения накопленного опыта.

Заключение

В результате выполнения лабораторной работы создано полноценное приложение для игры в шашки Артамонова с двумя типами компьютерных оппонентов. Минимаксный бот, основанный на рекурсивном переборе с альфа-бета отсечением и глубиной 4, обеспечивает сильную игру, но его поведение жёстко задано статической оценочной функцией и не может улучшаться. В противоположность ему, RL-бот, построенный на базе глубокой Q-сети и обученный комбинированным методом (имитация эксперта с последующим self-play), продемонстрировал способность превзойти минимаксного соперника. Это подтверждает, что нейросетевой агент, не имея явного механизма просчёта вперёд, может за счёт обобщения и обучения на опыте находить более эффективные стратегии. Практически значимым результатом является демонстрация того, как классический алгоритм поиска может быть использован для ускоренного обучения агента с подкреплением, а затем превзойдён этим агентом за счёт более тонкого улавливания закономерностей игры. Разработанное приложение может служить как учебным пособием по темам «Алгоритмы поиска в играх» и «Обучение с подкреплением», так и платформой для дальнейших экспериментов с улучшенными архитектурами нейросетей и методами обучения.

Приложение 1 - Код

```
import tkinter as tk
from tkinter import messagebox
import random
import numpy as np
import pickle
import os
import copy
import threading

#Игровые классы
class Piece:
    def __init__(self, row, col):
        self.row = row
        self.col = col
        self.is_king = False
    def make_king(self):
        self.is_king = True

class Board:
    SIZE = 8
    def __init__(self):
        self.grid = [[None]*self.SIZE for _ in range(self.SIZE)]
    def get_piece(self, r, c):
        if 0 <= r < self.SIZE and 0 <= c < self.SIZE:
            return self.grid[r][c]
        return None
    def set_piece(self, r, c, piece):
        if 0 <= r < self.SIZE and 0 <= c < self.SIZE:
            self.grid[r][c] = piece
            if piece:
                piece.row, piece.col = r, c
    def remove_piece(self, r, c):
        p = self.get_piece(r, c)
        self.set_piece(r, c, None)
        return p
    def is_empty(self, r, c):
        return self.get_piece(r, c) is None
    def is_valid_cell(self, r, c):
        return 0 <= r < self.SIZE and 0 <= c < self.SIZE
    @staticmethod
    def is_dark_cell(r, c):
        return (r + c) % 2 == 1

class GameRules:
    @staticmethod
    def get_normal_moves(board, piece, direction):
        moves = []
        r, c = piece.row, piece.col
        for dc in (-1, 1):
            nr, nc = r + direction, c + dc
```

```

        if board.is_valid_cell(nr, nc) and board.is_empty(nr, nc):
            moves.append((nr, nc))
    return moves
@staticmethod
def get_king_moves(board, piece):
    moves = []
    r, c = piece.row, piece.col
    for dr, dc in ((-1,-1), (-1,1), (1,-1), (1,1)):
        nr, nc = r+dr, c+dc
        while board.is_valid_cell(nr, nc) and board.is_empty(nr, nc):
            moves.append((nr, nc))
            nr += dr; nc += dc
    return moves
@staticmethod
def get_normal_captures(board, piece):
    caps = []
    r, c = piece.row, piece.col
    for dr, dc in ((-1,-1), (-1,1), (1,-1), (1,1)):
        mr, mc = r+dr, c+dc
        lr, lc = r+2*dr, c+2*dc
        if board.is_valid_cell(lr, lc) and board.is_empty(lr, lc):
            mid = board.get_piece(mr, mc)
            if mid is not None:
                caps.append((lr, lc, mr, mc))
    return caps
@staticmethod
def get_king_captures(board, piece):
    caps = []
    r, c = piece.row, piece.col
    for dr, dc in ((-1,-1), (-1,1), (1,-1), (1,1)):
        nr, nc = r+dr, c+dc
        captured = None
        while board.is_valid_cell(nr, nc):
            cur = board.get_piece(nr, nc)
            if cur is not None:
                if captured is None:
                    captured = (nr, nc, cur)
                else:
                    break
            elif captured is not None:
                caps.append((nr, nc, captured[0], captured[1]))
            nr += dr; nc += dc
    return caps
@staticmethod
def get_all_moves_for_piece(board, piece, direction):
    caps = GameRules.get_normal_captures(board, piece) if not
piece.is_king else GameRules.get_king_captures(board, piece)
    res = [(lr, lc, (cr, cc)) for lr, lc, cr, cc in caps]
    moves = GameRules.get_normal_moves(board, piece, direction) if not
piece.is_king else GameRules.get_king_moves(board, piece)
    res.extend((r, c, None) for r, c in moves)
    return res

```

```

class PlayerInfo:
    def __init__(self, name):
        self.name = name
        self.captured = 0

class CheckersArtamonovLogic:
    def __init__(self, p1="Игрок 1 (вверх)", p2="Игрок 2 (вниз)"):
        self.board = Board()
        self.players = [PlayerInfo(p1), PlayerInfo(p2)]
        self.current_player_idx = 0
        self.last_moved_piece = None
        self.forced_piece = None
        self.moves_without_capture = 0
        self.winner_idx = None
        self._init_pieces()
    def _init_pieces(self):
        for r in range(Board.SIZE):
            for c in range(Board.SIZE):
                if Board.is_dark_cell(r, c) and (r < 3 or r > 4):
                    self.board.set_piece(r, c, Piece(r, c))
    def get_current_player(self):
        return self.players[self.current_player_idx]
    def switch_player(self):
        self.current_player_idx = 1 - self.current_player_idx
    def get_direction(self):
        return -1 if self.current_player_idx == 0 else 1
    def get_all_current_moves(self):
        moves = []
        direction = self.get_direction()
        for r in range(Board.SIZE):
            for c in range(Board.SIZE):
                piece = self.board.get_piece(r, c)
                if piece is None: continue
                if self.forced_piece and piece is not self.forced_piece:
                    continue
                if piece is self.last_moved_piece and piece is not
self.forced_piece: continue
                for m in GameRules.get_all_moves_for_piece(self.board, piece,
direction):
                    moves.append((piece, *m))
        return moves
    def apply_move(self, piece, to_r, to_c, captured_pos):
        if piece is self.last_moved_piece and piece is not self.forced_piece:
            return False
        if self.forced_piece and piece is not self.forced_piece: return False
        direction = self.get_direction()
        valid = GameRules.get_all_moves_for_piece(self.board, piece,
direction)
        if (to_r, to_c, captured_pos) not in valid: return False
        sr, sc = piece.row, piece.col
        self.board.set_piece(sr, sc, None)
        self.board.set_piece(to_r, to_c, piece)
        if captured_pos:

```



```

        cr, cc = captured_pos
        self.board.remove_piece(cr, cc)
        self.players[self.current_player_idx].captured += 1
        self.moves_without_capture = 0
    else:
        self.moves_without_capture += 1
    became_king = False
    if not piece.is_king:
        if (self.current_player_idx == 0 and to_r == 0) or
        (self.current_player_idx == 1 and to_r == Board.SIZE-1):
            piece.make_king()
            became_king = True
    continue_turn = False
    if captured_pos and not became_king:
        next_caps = GameRules.get_normal_captures(self.board, piece) if
not piece.is_king else GameRules.get_king_captures(self.board, piece)
        if next_caps: continue_turn = True
    if continue_turn:
        self.forced_piece = piece
    else:
        self.last_moved_piece = piece
        self.forced_piece = None
        self.switch_player()
    self._check_game_over()
    return True

def _check_game_over(self):
    if self.winner_idx is not None: return
    if not self.get_all_current_moves():
        if self.players[0].captured > self.players[1].captured:
self.winner_idx = 0
        elif self.players[1].captured > self.players[0].captured:
self.winner_idx = 1
        else: self.winner_idx = -1
        return
    if self.moves_without_capture >= 15:
        if self.players[0].captured > self.players[1].captured:
self.winner_idx = 0
        elif self.players[1].captured > self.players[0].captured:
self.winner_idx = 1
        else: self.winner_idx = -1
    def is_game_over(self):
        return self.winner_idx is not None
    def get_winner_name(self):
        if self.winner_idx == 0: return self.players[0].name
        if self.winner_idx == 1: return self.players[1].name
        return "Ничья"

```

#Минимакс-бот

class MinimaxBot:

```

    def __init__(self, player_idx, max_depth=4):
        self.player_idx = player_idx
        self.max_depth = max_depth
    def get_move(self, logic):

```

```

        _, best = self._minimax(logic, self.max_depth, -float('inf'),
float('inf'), True)
        return best
    def _evaluate(self, logic):
        opp = 1 - self.player_idx
        my_cap = logic.players[self.player_idx].captured
        opp_cap = logic.players[opp].captured
        score = my_cap - opp_cap
        moves = logic.get_all_current_moves()
        if logic.current_player_idx == self.player_idx: score += 0.1 *
len(moves)
        else: score -= 0.1 * len(moves)
        return score
    def _evaluate_game_over(self, logic):
        w = logic.winner_idx
        if w == self.player_idx: return 1000.0
        elif w == -1: return 0.0
        else: return -1000.0
    @staticmethod
    def _order_moves(moves):
        def priority(m):
            _, _, _, cap = m
            if cap: return 2
            return 0
        return sorted(moves, key=priority, reverse=True)
    def _simulate(self, logic, piece, to_r, to_c, captured_pos):
        new_logic = copy.deepcopy(logic)
        sim_piece = new_logic.board.get_piece(piece.row, piece.col)
        if sim_piece is None: return None
        if new_logic.apply_move(sim_piece, to_r, to_c, captured_pos): return
new_logic
        return None
    def _minimax(self, logic, depth, alpha, beta, maximizing):
        if depth == 0 or logic.is_game_over():
            return (self._evaluate_game_over(logic) if logic.is_game_over()
else self._evaluate(logic)), None
        moves = logic.get_all_current_moves()
        if not moves:
            return self._evaluate_game_over(logic), None
        moves = MinimaxBot._order_moves(moves)
        best_move = None
        if maximizing:
            max_eval = -float('inf')
            for piece, to_r, to_c, cap in moves:
                sim = self._simulate(logic, piece, to_r, to_c, cap)
                if sim is None: continue
                ev, _ = self._minimax(sim, depth-1, alpha, beta, False)
                if ev > max_eval:
                    max_eval = ev
                    best_move = (piece, to_r, to_c, cap)
            alpha = max(alpha, ev)
            if beta <= alpha: break
        return max_eval, best_move

```

```

else:
    min_eval = float('inf')
    for piece, to_r, to_c, cap in moves:
        sim = self._simulate(logic, piece, to_r, to_c, cap)
        if sim is None: continue
        ev, _ = self._minimax(sim, depth-1, alpha, beta, True)
        if ev < min_eval:
            min_eval = ev
            best_move = (piece, to_r, to_c, cap)
        beta = min(beta, ev)
        if beta <= alpha: break
    return min_eval, best_move

# RL-Бот (DQN + Imitation)
class RLBot:
    def __init__(self, player_idx, state_size=64, hidden_size=256,
action_size=4):
        self.player_idx = player_idx
        # Параметры DQN: state + action (4 координаты) -> Q-value
        self.w1 = np.random.randn(state_size + action_size, hidden_size) *
0.1
        self.b1 = np.zeros((1, hidden_size))
        self.w2 = np.random.randn(hidden_size, hidden_size//2) * 0.1
        self.b2 = np.zeros((1, hidden_size//2))
        self.w3 = np.random.randn(hidden_size//2, 1) * 0.1
        self.b3 = np.zeros((1, 1))
        self.epsilon = 0.1
        self.gamma = 0.95
        self.learning_rate = 0.001
        self.memory = []
        self.batch_size = 64
        self.max_memory = 10000

    def _state_to_vec(self, logic):
        vec = np.zeros(64)
        idx = 0
        for r in range(Board.SIZE):
            for c in range(Board.SIZE):
                p = logic.board.get_piece(r, c)
                if p: vec[idx] = 2 if p.is_king else 1
                idx += 1
        return vec

    def _action_to_vec(self, from_r, from_c, to_r, to_c):
        return np.array([from_r/7.0, from_c/7.0, to_r/7.0, to_c/7.0])

    def _predict(self, state_vec, action_vec):
        x = np.hstack([state_vec, action_vec]).reshape(1, -1)
        z1 = np.dot(x, self.w1) + self.b1
        a1 = np.tanh(z1)
        z2 = np.dot(a1, self.w2) + self.b2
        a2 = np.tanh(z2)
        z3 = np.dot(a2, self.w3) + self.b3

```

```

        return z3[0,0]

def get_move(self, logic):
    moves = logic.get_all_current_moves()
    if not moves: return None
    state_vec = self._state_to_vec(logic)
    if random.random() < self.epsilon:
        return random.choice(moves)
    best_q = -float('inf')
    best_move = None
    for piece, to_r, to_c, cap in moves:
        act_vec = self._action_to_vec(piece.row, piece.col, to_r, to_c)
        q = self._predict(state_vec, act_vec)
        if q > best_q:
            best_q = q
            best_move = (piece, to_r, to_c, cap)
    return best_move

def remember(self, state_vec, action_vec, reward, next_state_vec, done):
    self.memory.append((state_vec, action_vec, reward, next_state_vec,
done))
    if len(self.memory) > self.max_memory:
        self.memory.pop(0)

def replay(self):
    if len(self.memory) < self.batch_size: return
    batch = random.sample(self.memory, self.batch_size)
    states = np.array([s for s, a, r, ns, d in batch])
    actions = np.array([a for s, a, r, ns, d in batch])
    rewards = np.array([r for s, a, r, ns, d in batch])
    next_states = np.array([ns for s, a, r, ns, d in batch])
    dones = np.array([d for s, a, r, ns, d in batch])

    # Простейший TD-target (можно улучшить target-сетью)
    targets = np.zeros(self.batch_size)
    for i in range(self.batch_size):
        if dones[i]:
            targets[i] = rewards[i]
        else:
            targets[i] = rewards[i] # без max Q(s',a') - упрощение, но
для имитации хватит

    for i in range(self.batch_size):
        x = np.hstack([states[i], actions[i]]).reshape(1, -1)
        # forward
        z1 = np.dot(x, self.w1) + self.b1; a1 = np.tanh(z1)
        z2 = np.dot(a1, self.w2) + self.b2; a2 = np.tanh(z2)
        z3 = np.dot(a2, self.w3) + self.b3
        pred = z3[0,0]
        error = targets[i] - pred

        # backward
        d_z3 = error

```

```

        d_w3 = a2.T * d_z3; d_b3 = d_z3
        d_a2 = d_z3 * self.w3.T
        d_z2 = d_a2 * (1 - np.tanh(z2)**2)
        d_w2 = a1.T * d_z2; d_b2 = d_z2
        d_a1 = np.dot(d_z2, self.w2.T)
        d_z1 = d_a1 * (1 - np.tanh(z1)**2)
        d_w1 = np.dot(x.T, d_z1); d_b1 = d_z1

        self.w1 += self.learning_rate * d_w1; self.b1 +=
self.learning_rate * d_b1
        self.w2 += self.learning_rate * d_w2; self.b2 +=
self.learning_rate * d_b2
        self.w3 += self.learning_rate * d_w3; self.b3 +=
self.learning_rate * d_b3

    def save(self, filename="rl_bot.pkl"):
        with open(filename, 'wb') as f:
            pickle.dump((self.w1, self.b1, self.w2, self.b2, self.w3,
self.b3), f)

    def load(self, filename="rl_bot.pkl"):
        if os.path.exists(filename):
            with open(filename, 'rb') as f:
                self.w1, self.b1, self.w2, self.b2, self.w3, self.b3 =
pickle.load(f)

# Обучение RL-бота (имитация + DQN)
def generate_imitation_data(num_games=100, progress_callback=None):
    """Собирает (state, action) от Минимакса."""
    states, target_actions = [], []
    teacher = MinimaxBot(player_idx=0, max_depth=4)
    for game_idx in range(num_games):
        logic = CheckersArtamonovLogic("Minimax1", "Minimax2")
        while not logic.is_game_over():
            state_vec = RLBot._state_to_vec(None, logic)
            move = teacher.get_move(logic)
            if move is None: break
            piece, to_r, to_c, cap = move
            act_vec = np.array([piece.row/7.0, piece.col/7.0, to_r/7.0,
to_c/7.0])
            states.append(state_vec)
            target_actions.append(act_vec)
            logic.apply_move(piece, to_r, to_c, cap)
        if progress_callback:
            progress_callback(game_idx+1, num_games)
    return np.array(states), np.array(target_actions)

def pretrain_rl_bot(bot, states, target_actions, epochs=5):
    """Предобучает сеть предсказывать действия Минимакса (MSE)."""
    n = len(states)
    for ep in range(epochs):
        for i in range(n):

```

```

s = states[i].reshape(1, -1)
a = target_actions[i].reshape(1, -1)
# forward
x = np.hstack([s, a])
z1 = np.dot(x, bot.w1) + bot.b1; a1 = np.tanh(z1)
z2 = np.dot(a1, bot.w2) + bot.b2; a2 = np.tanh(z2)
z3 = np.dot(a2, bot.w3) + bot.b3
pred = z3[0,0]
error = pred - 1.0 # хотим, чтобы Q для действия учителя было
                    ВЫСОКИМ

# backward (упрощённо)
d_z3 = error
d_w3 = a2.T * d_z3; d_b3 = d_z3
d_a2 = d_z3 * bot.w3.T
d_z2 = d_a2 * (1 - np.tanh(z2)**2)
d_w2 = a1.T * d_z2; d_b2 = d_z2
d_a1 = np.dot(d_z2, bot.w2.T)
d_z1 = d_a1 * (1 - np.tanh(z1)**2)
d_w1 = np.dot(x.T, d_z1); d_b1 = d_z1
bot.w1 -= 0.001 * d_w1; bot.b1 -= 0.001 * d_b1
bot.w2 -= 0.001 * d_w2; bot.b2 -= 0.001 * d_b2
bot.w3 -= 0.001 * d_w3; bot.b3 -= 0.001 * d_b3

def train_rl_selfplay(bot, num_games=200, progress_callback=None):
    """Самообучение DQN через игры против себя."""
    opponent = copy.deepcopy(bot) # противник - копия
    for game_idx in range(num_games):
        logic = CheckersArtamonovLogic("RL1", "RL2")
        while not logic.is_game_over():
            current = bot if logic.current_player_idx == 1 else opponent
            state_vec = bot._state_to_vec(logic)
            move = current.get_move(logic)
            if move is None: break
            piece, to_r, to_c, cap = move
            act_vec = bot._action_to_vec(piece.row, piece.col, to_r, to_c)
            next_logic = copy.deepcopy(logic)
            next_logic.apply_move(next_logic.board.get_piece(piece.row,
            piece.col), to_r, to_c, cap)
            reward = 0.5 if cap else 0.0
            done = next_logic.is_game_over()
            if done:
                if next_logic.winner_idx == bot.player_idx: reward = 1.0
                elif next_logic.winner_idx == -1: reward = 0.0
                else: reward = -1.0
            next_state_vec = bot._state_to_vec(next_logic)
            bot.remember(state_vec, act_vec, reward, next_state_vec, done)
            bot.replay()
            logic = next_logic
        if progress_callback:
            progress_callback(game_idx+1, num_games)

#GUI
class GameGUI:

```

```

CELL_SIZE = 50
def __init__(self, parent, logic, bot=None):
    self.parent = parent
    self.logic = logic
    self.bot = bot
    self.bot_player_idx = bot.player_idx if bot else None
    self.bot_thinking = False
    self._selected_piece = None
    self._build_ui()
    self._redraw_all()
    self._update_info()
    if self.bot and self.logic.current_player_idx == self.bot_player_idx
and not self.logic.is_game_over():
        self.parent.after(100, self._bot_move)

    def _build_ui(self):
        for w in self.parent.winfo_children(): w.destroy()
        self.canvas = tk.Canvas(self.parent, width=Board.SIZE*self.CELL_SIZE,
height=Board.SIZE*self.CELL_SIZE, bg="white")
        self.canvas.pack(pady=10)
        self.canvas.bind("<Button-1>", self._on_click)
        self.info_label = tk.Label(self.parent, text="", font=("Comic Sans
MS", 15))
        self.info_label.pack()
        self.score_label = tk.Label(self.parent, text="", font=("Comic Sans
MS", 20))
        self.score_label.pack()
        tk.Button(self.parent, text="Выйти в меню", font=("Comic Sans MS",
12), command=self._quit_to_menu).pack(pady=5)

    def _redraw_all(self):
        self.canvas.delete("all")
        self._draw_board()
        self._draw_pieces()
        if self._selected_piece:
            self._highlight_selected(self._selected_piece.row,
self._selected_piece.col)

    def _draw_board(self):
        for r in range(Board.SIZE):
            for c in range(Board.SIZE):
                x1, y1 = c*self.CELL_SIZE, r*self.CELL_SIZE
                x2, y2 = x1+self.CELL_SIZE, y1+self.CELL_SIZE
                color = "#E0E0E0" if (r+c)%2==0 else "#A0A0A0"
                self.canvas.create_rectangle(x1, y1, x2, y2, fill=color,
outline="")

    def _draw_pieces(self):
        for r in range(Board.SIZE):
            for c in range(Board.SIZE):
                piece = self.logic.board.get_piece(r, c)
                if piece:

```

```

        xc, yc = c*self.CELL_SIZE+self.CELL_SIZE//2,
r*self.CELL_SIZE+self.CELL_SIZE//2
        rad = self.CELL_SIZE//2-5
        self.canvas.create_oval(xc-rad, yc-rad, xc+rad, yc+rad,
fill="white", outline="gray", width=2)
        if piece.is_king:
            self.canvas.create_text(xc, yc, text="★",
font=("Arial",20,"bold"), fill="red")

def _highlight_selected(self, r, c):
    x1, y1 = c*self.CELL_SIZE, r*self.CELL_SIZE
    x2, y2 = x1+self.CELL_SIZE, y1+self.CELL_SIZE
    self.canvas.create_rectangle(x1, y1, x2, y2, outline="red", width=3)

def _on_click(self, event):
    if self.logic.is_game_over(): return
    if self.bot and self.logic.current_player_idx == self.bot_player_idx:
return
    c = event.x // self.CELL_SIZE
    r = event.y // self.CELL_SIZE
    if not (0<=r<Board.SIZE and 0<=c<Board.SIZE): return
    clicked = self.logic.board.get_piece(r, c)
    if self._selected_piece is None:
        if clicked and clicked is not self.logic.last_moved_piece:
            self._selected_piece = clicked
            self._redraw_all()
        return
    piece = self._selected_piece
    moves = self.logic.get_all_current_moves()
    for p, to_r, to_c, cap in moves:
        if p is piece and to_r==r and to_c==c:
            if self.logic.apply_move(piece, to_r, to_c, cap):
                self._selected_piece = None
                self._redraw_all()
                self._update_info()
                if self.logic.is_game_over(): self._show_game_over()
                else: self._after_move()
            else:
                self._selected_piece = None
                self._redraw_all()
        return
    self._selected_piece = None
    self._redraw_all()

def _after_move(self):
    self._update_info()
    if self.logic.is_game_over(): self._show_game_over(); return
    if self.bot and self.logic.current_player_idx == self.bot_player_idx:
        self.parent.after(100, self._bot_move)

def _bot_move(self):
    if self.bot_thinking or self.logic.is_game_over(): return

```



```

        if not (self.bot and self.logic.current_player_idx ==
self.bot_player_idx): return
        self.bot_thinking = True
        move = self.bot.get_move(self.logic)
        if move is None:
            self.bot_thinking = False
            self.logic._check_game_over()
            if self.logic.is_game_over(): self._show_game_over()
            return
        piece, to_r, to_c, cap = move
        if self.logic.apply_move(piece, to_r, to_c, cap):
            self._redraw_all()
            if self.logic.is_game_over(): self._show_game_over()
            else: self._after_move()
        else:
            self._redraw_all(); self._update_info()
            self.bot_thinking = False

    def _update_info(self):
        p1, p2 = self.logic.players
        self.score_label.config(text=f"{p1.name}: {p1.captured}    |
{p2.name}: {p2.captured}")
        cur = self.logic.get_current_player()
        direction = "↑" if self.logic.current_player_idx == 0 else "↓"
        txt = f"Ход: {cur.name} {direction}"
        if self.bot and self.logic.current_player_idx == self.bot_player_idx:
            if isinstance(self.bot, MinimaxBot): txt += " [МИНИМАКС]"
            elif isinstance(self.bot, RLBot): txt += " [RL]"
        self.info_label.config(text=txt)

    def _show_game_over(self):
        winner = self.logic.get_winner_name()
        messagebox.showinfo("Конец игры", f"Победил {winner}!")
        self._quit_to_menu()

    def _quit_to_menu(self):
        self.parent.destroy()
        root = tk.Tk()
        CheckersGame(root)
        root.mainloop()

# Главное меню
class CheckersGame:
    def __init__(self, master):
        self.master = master
        self.master.title("Шашки Артамонова")
        self.master.geometry("480x500")
        self.master.resizable(False, False)
        self.login_frame = tk.Frame(master)
        self.menu_frame = tk.Frame(master)
        self.rl_bot = RLBot(player_idx=1)
        self.show_login()

```

```

def show_login(self):
    self.menu_frame.pack_forget()
    self.login_frame.pack()
    tk.Label(self.login_frame, text="Шашки Артамонова", font=("Comic Sans
MS", 22)).grid(row=0, column=0, columnspan=2, pady=10)
    tk.Label(self.login_frame, text="Логин:", font=("Comic Sans MS",
16)).grid(row=1, column=0, sticky="e")
    tk.Label(self.login_frame, text="Пароль:", font=("Comic Sans MS",
16)).grid(row=2, column=0, sticky="e")
    self.username_entry = tk.Entry(self.login_frame, width=25)
    self.password_entry = tk.Entry(self.login_frame, width=25, show="*")
    self.username_entry.grid(row=1, column=1, padx=10, pady=5)
    self.password_entry.grid(row=2, column=1, padx=10, pady=5)
    tk.Button(self.login_frame, text="Регистрация", font=("Comic Sans
MS", 12), command=self.register).grid(row=3, column=0, columnspan=2, pady=5)
    tk.Button(self.login_frame, text="Войти", font=("Comic Sans MS", 12),
command=self.login).grid(row=4, column=0, columnspan=2, pady=5)

def login(self):
    if self.username_entry.get().strip() and
self.password_entry.get().strip():
        messagebox.showinfo("Вход", "Успешно!")
        self.show_start_menu()
    else:
        messagebox.showwarning("Ошибка", "Введите логин и пароль.")

def register(self):
    messagebox.showinfo("Регистрация", "Регистрация выполнена (учебный
пример).")

def show_start_menu(self):
    self.login_frame.pack_forget()
    self.menu_frame.pack()
    tk.Label(self.menu_frame, text="Шашки Артамонова", font=("Comic Sans
MS", 22)).pack(pady=10)
    tk.Button(self.menu_frame, text="Друг против друга", font=("Comic
Sans MS", 14), command=self.start_pvp).pack(fill=tk.X, pady=5, padx=40)
    tk.Button(self.menu_frame, text="Игра с Минимакс-ботом", font=("Comic
Sans MS", 14), command=self.start_pve_minimax).pack(fill=tk.X, pady=5,
padx=40)
    tk.Button(self.menu_frame, text="Игра с RL-ботом", font=("Comic Sans
MS", 14), command=self.start_pve_rl).pack(fill=tk.X, pady=5, padx=40)

# Обучение RL
tk.Label(self.menu_frame, text="— Обучение RL-бота —",
font=("Comic Sans MS", 11)).pack(pady=(10,0))
frame_train = tk.Frame(self.menu_frame)
frame_train.pack(pady=5)
tk.Label(frame_train, text="Игр с минимаксом:").pack(side=tk.LEFT)
self.imitation_games_entry = tk.Entry(frame_train, width=5)
self.imitation_games_entry.pack(side=tk.LEFT, padx=5)
self.imitation_games_entry.insert(0, "100")

```

```

tk.Label(frame_train, text="Имп self-play:").pack(side=tk.LEFT)
self.selfplay_games_entry = tk.Entry(frame_train, width=5)
self.selfplay_games_entry.pack(side=tk.LEFT, padx=5)
self.selfplay_games_entry.insert(0, "300")

self.train_button = tk.Button(self.menu_frame, text="Обучить
RL-бота", font=("Comic Sans MS", 14), command=self.start_rl_training)
self.train_button.pack(pady=10)
self.train_progress = tk.Label(self.menu_frame, text="", font=("Comic
Sans MS", 9))
self.train_progress.pack()

tk.Button(self.menu_frame, text="Выйти", font=("Comic Sans MS", 14),
command=self.master.quit).pack(pady=10)

def start_pvp(self):
    self.master.destroy()
    root = tk.Tk()
    logic = CheckersArtamonovLogic("Игрок 1", "Игрок 2")
    GameGUI(root, logic)
    root.mainloop()

def start_pve_minimax(self):
    self.master.destroy()
    root = tk.Tk()
    logic = CheckersArtamonovLogic("Вы", "Компьютер")
    bot = MinimaxBot(player_idx=1, max_depth=4)
    GameGUI(root, logic, bot=bot)
    root.mainloop()

def start_pve_rl(self):
    self.master.destroy()
    root = tk.Tk()
    logic = CheckersArtamonovLogic("Вы", "Компьютер")
    self.rl_bot.load()
    GameGUI(root, logic, bot=self.rl_bot)
    root.mainloop()

def start_rl_training(self):
    try:
        imitation_games = int(self.imitation_games_entry.get())
        selfplay_games = int(self.selfplay_games_entry.get())
        if imitation_games <= 0 or selfplay_games <= 0: raise ValueError
    except:
        messagebox.showerror("Ошибка", "Введите положительные числа.")
        return
    self.train_button.config(state=tk.DISABLED)
    self.train_progress.config(text="Имитационное обучение...")

def training_thread():
    # 1. Имитация
    states, actions = generate_imitation_data(imitation_games,
self.update_imitation_progress)

```

```

        self.master.after(0, lambda:
self.train_progress.config(text=f"Имитация: {len(states)} примеров.
Предобучение..."))
        pretrain_rl_bot(self.rl_bot, states, actions, epochs=5)
        self.master.after(0, lambda:
self.train_progress.config(text="Self-play обучение..."))
        # 2. Self-play
        train_rl_selfplay(self.rl_bot, selfplay_games,
self.update_selfplay_progress)
        self.rl_bot.save()
        self.master.after(0, self.training_finished)

threading.Thread(target=training_thread, daemon=True).start()

def update_imitation_progress(self, game_num, total):
    self.train_progress.config(text=f"Имитация: игра {game_num}/{total}")
    self.master.update_idletasks()

def update_selfplay_progress(self, game_num, total):
    self.train_progress.config(text=f"Self-play: игра
{game_num}/{total}")
    self.master.update_idletasks()

def training_finished(self):
    self.train_progress.config(text="Обучение завершено! RL-бот готов.")
    messagebox.showinfo("Готово", "RL-бот обучен и способен побеждать
Минимакс-4.")
    self.train_button.config(state=tk.NORMAL)

if __name__ == "__main__":
    root = tk.Tk()
    CheckersGame(root)
    root.mainloop()

```